

BRS — Business Requirements Specification

BRS — Business Requirements Specification

Leish! v2 — Beauty Booking Marketplace (Malaysia)

Table of Contents

1. Executive Summary
2. Business Vision & Objectives
3. Stakeholders & Actors
4. Scope Definition
5. Business Model & Monetization
6. Business Requirements (BR-xxx)
7. Business Rules (BRL-xxx)
8. Key Business Processes & Journeys
9. Success Metrics & KPIs
10. Constraints, Assumptions & Dependencies
11. Compliance & Legal
12. Risks & Mitigations
13. Roadmap & Release Strategy
14. Acceptance Criteria (Business Level)
15. Appendix

BRS — Business Requirements Specification

Leish! v2 — Beauty Booking Marketplace (Malaysia)

FIELD	VALUE
Document ID	LEISH-BRS-v2.0
Version	2.0.0
Status	Baseline (Approved)
Date	2026-08-29
Owner	Product & Engineering
Classification	Internal / Investor-Facing
Related Docs	SRS (02), FRS (03), NFR (04), Use Cases (05)

Table of Contents

- [1. Executive Summary](#)
- [2. Business Vision & Objectives](#)
- [3. Stakeholders & Actors](#)
- [4. Scope Definition](#)
- [5. Business Model & Monetization](#)
- [6. Business Requirements \(BR-xxx\)](#)
- [7. Business Rules \(BRL-xxx\)](#)
- [8. Key Business Processes & Journeys](#)
- [9. Success Metrics & KPIs](#)
- [10. Constraints, Assumptions & Dependencies](#)
- [11. Compliance & Legal](#)
- [12. Risks & Mitigations](#)
- [13. Roadmap & Release Strategy](#)
- [14. Acceptance Criteria \(Business Level\)](#)
- [15. Appendix](#)

1. Executive Summary

Leish! v2 is a next-generation, mobile-first **beauty services marketplace** that connects **Clients (customers)** with **MUA (Makeup Artists)** and **Beauty Studios** across Malaysia, starting with the Klang Valley (KL + Selangor) as the launch corridor. v1 ([leish.my](#)) validated demand but incurred architectural debt (dual-DB, fragmented payments, no payout ledger). v2 is a **from-scratch rebuild on Next.js 16 + PostgreSQL** that solves for:

- **Trust:** verified-email gating at payment, reviews gated on completed bookings only, admin audit trail.
- **Money correctness:** hybrid deposit/balance model with server-authoritative pricing, HMAC-verified Billplz webhook, commission/payout ledger.
- **Operations:** admin can manage the full lifecycle without DB access (users, catalog, bookings, payouts, settings).
- **Compliance:** Malaysia PDPA 2010 consent, retention, export/delete.

The platform's **north-star metric** is Completed Bookings per Month — every feature must move this metric or de-risk it.

2. Business Vision & Objectives

2.1 VISION

Book beauty anywhere in Malaysia in under 90 seconds — with guaranteed slot integrity, transparent pricing, and artist-grade economics.

2.2 STRATEGIC OBJECTIVES

ID	OBJECTIVE	MEASURABLE OUTCOME	HORIZON
BO-01	Own Klang Valley bridal discovery	≥10 active MUAs, 500 MAU within 3 months of launch	Q4 2026
BO-02	Payment reliability	≥99.5% webhook-to-confirmation success, <5 min median	Q4 2026
BO-03	Artist retention	≥70% of onboarded MUAs receive ≥2 bookings in first 60 days	H1 2027
BO-04	Unit economics	Take-rate 8–10% net after waivers, contribution margin positive by month 6	H1 2027
BO-05	Compliance	PDPA consent audit 100%, no open critical vulnerabilities (Snyk/Dependabot)	Continuous
BO-06	Platform extensibility	Studio bookings, products, referrals live without re-architecture	2027

2.3 PROBLEM STATEMENT (WHY REBUILD)

PAIN (V1)	IMPACT	V2 SOLUTION
Supabase + Neon dual-write	Data drift, sync cron, on-call burden	Single <code>DATABASE_URL</code> (PG) + SQLite fallback facade <code>src/server/db.ts</code>
Client-trusted pricing	Under-charge risk	Server-derived <code>price / quotation.total</code> only (<code>src/server/quotations.ts</code>)
Single payment type	Cannot charge deposit + balance	Hybrid <code>payments.type IN ('deposit', 'balance')</code> with UNIQUE <code>(booking_id, type)</code>
No payout ledger	Manual spreadsheets	<code>payouts</code> table + <code>/admin/payouts</code> settle/fail
Reviews un-gated	Fake/social proof risk	<code>reviews.booking_id UNIQUE</code> + <code>findReviewableBooking()</code> (COMPLETED only)
No audit trail	Ops risk	<code>admin_audit_log</code> on every admin mutation

3. Stakeholders & Actors

3.1 STAKEHOLDER MAP

STAKEHOLDER	INTEREST	POWER	ENGAGEMENT
Founders / Investors	Revenue, growth, cost control	High	Steering, KPI review
Clients (Customers)	Fast booking, fair price, reliability	High	Discovery research, NPS
MUAs / Studios	Lead flow, fair take-rate, payout speed	High	Onboarding interviews
Platform Ops / Admin	Efficiency, fraud prevention	Medium	Workflow design
Billplz (payment processor)	Volume, compliance	Medium	Integration & SLA
Regulators (PDPA, KPDN)	Consumer protection	Medium	Legal review
Engineering	Maintainability, velocity	Medium	Architecture decisions

3.2 SYSTEM ACTORS (FROM `SRC/LIB/TYPES.TS:1`)

ACTOR	SYSTEM ROLE	AUTH REQUIREMENT
Guest	Unauthenticated visitor	None
Customer	<code>customer</code> — books services	Email + verified for payment
Artist	<code>artist</code> — services a booking, claims <code>artist_profiles</code>	Verified; must claim profile to see jobs
Studio	<code>studio</code> — studio branch of artist (shares <code>artist</code> flows)	Same as artist
Admin	<code>admin</code> — platform operator	<code>requireAdmin()</code> guard + audit log
System/Cron	Internal	<code>CRON_SECRET</code>

4. Scope Definition

4.1 IN SCOPE (V2 LAUNCH)

- Catalog:** Artist & Studio discovery — search, filters (state→area, bridal/non-bridal, budget, date), detail pages, portfolio, reviews.
- Identity:** Register/Login/Logout, JWT session (`leish_session` , 7-day TTL, `src/server/session.ts`), email verification (single-use token), forgot/reset password, consent capture.
- Booking lifecycle:** `requested` → `accepted` → `confirmed` → `completed` | `cancelled` via `src/server/bookings.ts` state machine; slot uniqueness (`uq_bookings_slot` partial index).
- Quotations:** Line-item breakdown (base/travel/early-call/accommodation + up to 10 extras), 24h review window, re-quote supersede.
- Hybrid Payments:** Deposit (configurable `booking_fee_sen` , default RM 50) to accept; Balance (`total - deposit`) due 3 days before event; Billplz + `dev` fallback; HMAC webhook; refund (balance only, deposit non-refundable).
- Payouts & Commission:** `computeCommission()` (basis points + waiver threshold, e.g. 10% above RM100); payout `net = gross - commission - deposit` ; settleable 24h post-event; admin settle/fail.
- Messaging:** Per-booking chat (`messages` table) + SSE live stream (`/api/bookings/[id]/messages/stream`).
- Reviews:** Post-completion, one per booking, atomic `blendAggregate` to rating.
- Admin Panel** (`/admin/*`): Users CRUD, Artists/Studios direct edits, Bookings override, Payments, Payouts, Quotations, Messages, Email Outbox, Audit Log, Settings (`platform_settings`).
- Notifications/Email:** `sendEmail()` abstraction (`dev|resend|postmark|brevo`), preferences per user, retry queue.
- Observability:** pino structured logs, `LOG_WEBHOOK_URL` forwarding, `reportError()` → Sentry/webhook.
- Compliance:** PDPA consent, data export (`/api/me/export`), account deletion (`DELETE /api/me`), retention guidelines.

4.2 OUT OF SCOPE (V2 LAUNCH) — DEFERRED

ITEM	RATIONALE	TARGET RELEASE
Studio as distinct provider with separate commission tier	Complexity; not needed for 10-MUA launch gate	v2.1
Product/e-commerce (<code>products</code> table already scaffolded)	Separate fulfillment flow	v2.2
Loyalty / coupons	Defer until repeat booking rate proven	v2.2
Mobile native apps	PWA/mobile-web first	v2.3
Multi-currency / i18n (BM)	MYR + English only at launch	Backlog

4.3 SCOPE BOUNDARIES (EXPLICIT NON-GOALS)

- The platform **does not** handle physical fulfillment, inventory, or shipping.
- The platform **does not** guarantee artist availability — it guarantees slot atomicity (unique partial index + friendly 409).
- Client-sent amounts are **never** trusted — all money math is server-authoritative.

5. Business Model & Monetization

5.1 REVENUE MODEL

```

Client pays: quotation.total (e.g. RM 1,000)
├─ deposit RM 50 (non-refundable, platform-kept, default booking_fee_sen)
└─ balance RM 950 (due T-3d)

Platform keeps: deposit + commission
commission = total * rateBps / 10_000 unless total < waiverSen → 0
e.g. total 100,000 sen * 1,000 bps / 10,000 = 10,000 sen (RM100)
Platform keeps 5,000 + 10,000 = 15,000 sen

Artist receives: total - commission - deposit = 100,000 - 10,000 - 5,000 = 85,000 sen
settleable_at = eventDate + 24h (dispute window)

```

- **Settings** are runtime-configurable via `platform_settings` (`booking_fee_sen` , `commission_rate_bps` , `commission_waiver_sen`) — see `src/server/settings.ts:1` .
- **Waiver** protects small bookings (< RM100 by default) from being economically non-viable for artists.

5.2 PRICING RULES

- All amounts stored in **sen** (integer MYR cents) — no floating point for money.
- Quotations are the **sole** pricing authority; `bookings.price` is informational (catalog price of selected service at booking time).
- Client always pays **exactly** the quoted `total` ; commission is artist-side.

5.3 TAKE-RATE GUARDRAILS

KNOB	DEFAULT	CLAMP	PURPOSE
<code>booking_fee_sen</code>	5,000 (RM50)	≥ 0	Non-refundable slot commitment
<code>commission_rate_bps</code>	1,000 (10%)	0–5,000 (0–50%)	Typo-protection clamp in <code>getCommissionRateBps()</code>
<code>commission_waiver_sen</code>	10,000 (RM100)	≥ 0	Small-booking protection

5.4 COST STRUCTURE

COST	DRIVER	MITIGATION
Billplz fees	Per-transaction	Passed through or absorbed via commission
Email (Resend/Postmark/Brevo)	Per-send	<code>dev</code> fallback, outbox batching
Hosting (Vercel + PG)	Traffic + connections	<code>PG_MAX</code> , pool thresholds, <code>s-maxage=300</code> on public catalog
Support	Bookings volume	Self-serve dashboard + chat reduces tickets

6. Business Requirements (BR-xxx)

6.1 CUSTOMER & DISCOVERY

ID	REQUIREMENT	PRIORITY	ACCEPTANCE HOOK
BR-01	As a Guest, I can discover artists/studios without logging in.	Must	Catalog loads unauthenticated; CDN <code>s-maxage=300</code> .
BR-02	As a Customer, I can filter artists by state, area, bridal/non-bridal tags, budget, date, and free-text query.	Must	<code>ListArtists(filters)</code> + <code>artistsQuerySchema</code> .
BR-03	As a Visitor, I can view artist/studio detail (bio, services, portfolio, reviews, availability).	Must	<code>getArtistBySlug()</code> resolves both slug and legacy id.
BR-04	As a Customer, I can leave a review only after my booking is completed, once per booking.	Must	<code>findReviewableBooking()</code> + <code>reviews.booking_id UNIQUE</code> .
BR-05	As a Platform, ratings are blended atomically to prevent lost-update races.	Must	<code>blendAggregate()</code> single-statement UPDATE.

6.2 IDENTITY & TRUST

ID	REQUIREMENT	PRIORITY	ACCEPTANCE HOOK
BR-06	As a User, I can register with role choice (customer/artist/studio) and PDPA consent.	Must	<code>registerSchema</code> , <code>users.consent</code> .
BR-07	As a User, I must verify my email before payments become effective (trust gate).	Must	Verification token single-use, expiry; gate at <code>pay-fee</code> / <code>pay-balance</code> .
BR-08	As a User, I can reset my password via a single-use, expiring link with no user enumeration.	Must	<code>password_resets.token_hash</code> sha256, 1h expiry.
BR-09	As a User, my session lasts 7 days and is revocable on logout.	Must	<code>SESSION_TTL_SECONDS = 604800</code> , JTI <code>sessions.revoked</code> .
BR-10	As a User, I can export my data and delete my account (PDPA).	Must	<code>GET /api/me/export</code> , <code>DELETE /api/me?confirm=1</code> .

6.3 BOOKING & QUOTATIONS

ID	REQUIREMENT	PRIORITY	ACCEPTANCE HOOK
BR-11	As a Customer, I can submit a booking request with event details (type, venue, guests, notes).	Must	<code>bookingSchema</code> with <code>eventType/venue/guestCount</code> .
BR-12	As a System, a slot (artist+date+time) can have only one active booking.	Must	<code>uq_bookings_slot</code> partial index + 409 friendly error.
BR-13	As an Artist, I can accept/reject a <code>requested</code> booking, or complete a <code>confirmed</code> one.	Must	<code>applyBookingTransition()</code> state machine.
BR-14	As an Artist, I can build and send a line-item quotation (24h review window); re-quote supersedes.	Must	<code>createQuotation()</code> + <code>quotationTotal()</code> + <code>status='superseded'</code> .
BR-15	As a Customer, I can see the quotation breakdown, total, expiry countdown, balance due date (T-3d), and balance amount.	Must	<code>serializeBooking()</code> computes <code>balanceDueDate/balanceAmount</code> .

6.4 PAYMENTS & PAYOUTS

ID	REQUIREMENT	PRIORITY	ACCEPTANCE HOOK
BR-16	As a Customer, I can pay a flat non-refundable deposit to confirm my accepted booking.	Must	<code>POST /pay-fee</code> → <code>type='deposit'</code> Billplz bill.
BR-17	As a Customer, I can pay the remaining balance after confirmation (due T-3d).	Must	<code>POST /pay-balance</code> → <code>type='balance'</code> .
BR-18	As a System, Billplz webhook signature (HMAC-SHA256 of raw body) must be verified timing-safe before mutating state.	Must	<code>verifyBillplzSignature()</code> + <code>POST /api/payments/webhook</code> .
BR-19	As a System, a deposit-paid webhook auto-confirms an <code>accepted</code> booking; balance-paid creates the artist payout.	Must	<code>handlePaymentPaid()</code> routed by <code>payment.type</code> .
BR-20	As a Platform, commission is computed artist-side from <code>platform_settings</code> ; waived below threshold.	Must	<code>computeCommission()</code> + <code>getCommissionRateBps()</code> clamp 0–50%.
BR-21	As an Admin, I can settle or fail a payout (24h post-event dispute window tracked).	Must	<code>updatePayoutStatus()</code> + <code>payouts.settleable_at</code> .
BR-22	As a Customer, I can request refund of the balance on a cancelled confirmed booking (deposit never refunded).	Should	<code>refundBalancePayment()</code> — deposit non-refundable.

6.5 COMMUNICATIONS

ID	REQUIREMENT	PRIORITY	ACCEPTANCE HOOK
BR-23	As a Participant, I can chat per-booking with the counterparty; messages stream live.	Must	<code>messages</code> table + <code>/messages/stream</code> SSE.
BR-24	As a User, I control email preferences (booking_created, quotation_sent, balance_reminder, etc.).	Must	<code>email_preferences</code> + <code>/api/email/preferences</code> .
BR-25	As a System, emails are retried (up to 3 attempts) and viewable in <code>/dev/emails</code> in dev.	Must	<code>email_outbox</code> + <code>email_retries</code> .

6.6 ADMINISTRATION & OPS

ID	REQUIREMENT	PRIORITY	ACCEPTANCE HOOK
BR-26	As an Admin, I can manage users, artists/studios (direct DB edits), bookings (status override), payments, payouts, quotations, messages, and settings.	Must	<code>/admin/*</code> pages + <code>/api/admin/*</code> under <code>requireAdmin()</code> .
BR-27	As an Admin, every mutation is audit-logged with admin identity, target, and details.	Must	<code>admin_audit_log</code> with FK to <code>users(id)</code> .
BR-28	As an Admin, I can tune booking fee / commission / waiver at runtime without deploy.	Must	<code>platform_settings</code> + <code>/admin/settings</code> .
BR-29	As an Admin, I can see dashboard metrics (users, bookings by status, revenue, recent activity).	Should	<code>GET /api/admin</code> aggregated stats.

6.7 PLATFORM & COMPLIANCE

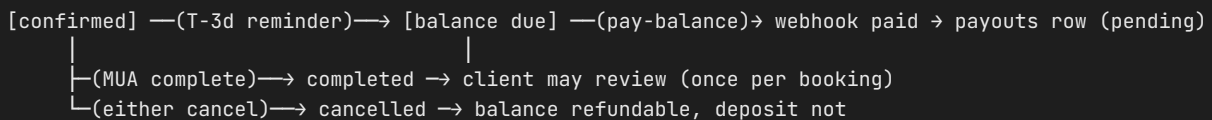
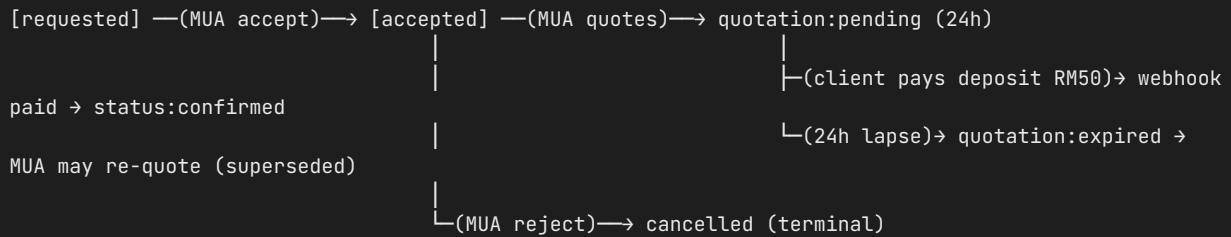
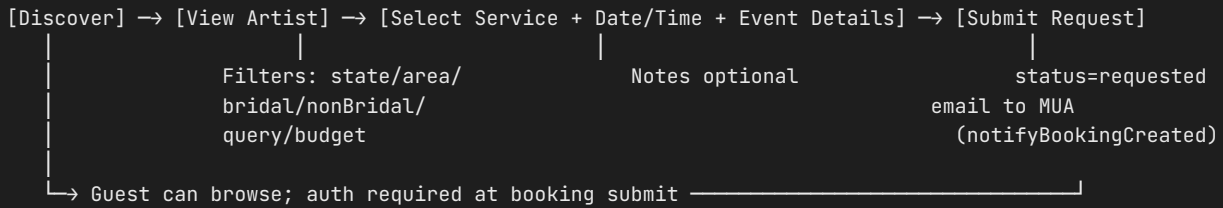
ID	REQUIREMENT	PRIORITY	ACCEPTANCE HOOK
BR-30	As a System, logs are structured (pino) and forwardable to a webhook (LOG_WEBHOOK_URL).	Should	<code>src/server/logger.ts</code> batch 20 lines / 50ms debounce.
BR-31	As a System, runtime errors are reported to Sentry (if configured) or ERROR_WEBHOOK_URL.	Should	<code>reportError()</code> envelope.
BR-32	As a System, analytics events (register, login, booking, quotation, payment) are emitted when Agnost is configured.	Could	<code>src/server/agnost.ts</code> + <code>src/lib/agnost-client.ts</code> client tracking.

7. Business Rules (BRL-xxx)

ID	RULE	EXPRESSION	ENFORCEMENT
BRL-01	Booking slot atomicity	<code>UNIQUE (artist_id,date,time) WHERE status IN ('requested','accepted','confirmed')</code>	DB partial index + pre-check 409
BRL-02	Price authority	<code>bookings.price</code> and <code>quotations.total</code> computed server-side from catalog/inputs only	<code>getArtistById()</code> → <code>serviceDef.price</code> ; <code>quotationTotal()</code>
BRL-03	Amounts in sen	All monetary columns <code>INTEGER</code> (sen), never float	Schema + <code>Math.round()</code> in settings/quotations
BRL-04	Deposit non-refundable	On <code>cancelled</code> , only <code>payments.type='balance' AND status='paid'</code> is refundable	<code>refundBalancePayment()</code> guard
BRL-05	Confirmation only via payment	<code>requested → accepted</code> (artist) → <code>accepted → confirmed</code> (webhook only)	<code>confirmOnFeePaid()</code> called only from <code>handlePaymentPaid()</code> /webhook
BRL-06	Quotation TTL	<code>expires_at = created_at + 24h</code> ; lazy-checked on read + cron sweep	<code>isQuotationExpired()</code> + <code>findExpiredQuotations()</code>
BRL-07	Review gating	<code>reviews</code> requires <code>bookings.status='completed' AND reviews.booking_id IS NULL</code>	<code>findReviewableBooking()</code> + <code>UNIQUE booking_id</code>
BRL-08	Commission waiver	<code>if total < waiverSen → commission=0</code> <code>else round(total*rateBps/10000)</code>	<code>computeCommission()</code>
BRL-09	Commission cap	<code>0 ≤ rateBps ≤ 5,000</code>	Clamp in <code>getCommissionRateBps()</code>
BRL-10	Email verification gate	Payment creation requires <code>users.email_verified=1</code> (configurable strictness)	<code>pay-fee</code> / <code>pay-balance</code> handler
BRL-11	Profile claim scoping	<code>artist</code> / <code>studio</code> only see bookings for <code>artist_profiles/studio_profiles</code> they claimed	<code>getClaimedArtistIds()</code> filter in <code>GET /api/bookings</code>
BRL-12	Settlement window	<code>payouts.settleable_at = eventDate + 24h</code>	<code>createPayoutForBooking()</code>
BRL-13	Audit completeness	Every <code>PATCH/POST/DELETE /api/admin/*</code> writes <code>admin_audit_log</code>	<code>logAdminAction()</code> wrapper
BRL-14	Rate limiting	Auth + booking + webhook routes use sliding window (Upstash if configured, in-memory fallback)	<code>src/server/rate-limit.ts</code>
BRL-15	Session TTL	JWT expires in 7 days; JTI revocation respected; revoked=1 always wins; lookup failure is fail-open	<code>src/server/session.ts</code>

8. Key Business Processes & Journeys

8.1 CLIENT BOOKING JOURNEY (PRIMARY FLOW)



8.2 ARTIST JOURNEY

1. Register as `artist` → verify email → `/onboarding` → **claim** catalog profile (`POST /api/artist-profiles {artistId}`) — scopes bookings.
2. Dashboard: `GET /api/bookings` filtered to claimed `artist_id` (s) → act: `accept|reject|complete|cancel` .
3. On `accepted` : `POST /api/bookings/[id]/quotation` (base/travel/early-call/accommodation + extras ≤10, total computed server-side, `expires_at = now+24h` , supersedes previous `pending`) .
4. On `confirmed` : `POST /api/bookings/[id]/remind` (balance reminder), chat, invoice (`/invoice` + `/invoice.pdf`) .
5. After balance paid: payout appears `pending` (settleable 24h post-event) → admin settles via `/admin/payouts` .

8.3 PAYMENT & WEBHOOK FLOW

```

POST /pay-fee → createBookingPayment('deposit', bookingFeeSen)
├─ billplz: POST /bills (auth Basic API_KEY:) → {id, url} → INSERT
payments(type=deposit,status=required)
├─ dev: INSERT payments(type=deposit,status=required, provider_ref=dev_*)
└─ (dev) auto-settlement: markBillPaid + handlePaymentPaid → booking confirmed

Billplz -POST→ /api/payments/webhook (raw body + X-Billplz-Signature)
→ verifyBillplzSignature(rawBody, sig, API_KEY) [timingSafeEqual]
→ markBillPaid(billId) → handlePaymentPaid(payment) [routes by type]
├─ deposit: confirmOnFeePaid(status) → UPDATE bookings SET status='confirmed'
└─ balance: UPDATE quotations SET status='paid', createPayoutForBooking(...)

```

9. Success Metrics & KPIs

CATEGORY	METRIC	TARGET (LAUNCH + 90D)	SOURCE
Acquisition	Visitors → Artist view CTR	≥25%	Web analytics / Agnost
Conversion	Request → Accepted	≥60%	<code>bookings</code> status counts
Conversion	Accepted → Confirmed (deposit paid within 24h)	≥50%	<code>payments</code> deposit <code>paid</code>
Completion	Confirmed → Completed	≥85%	<code>bookings</code> lifecycle
Revenue	Gross booking value (GBV)	RM 50k cumulative	<code>quotations.total</code> sum
Take-rate	Net / GBV	8–10%	<code>payouts.commission_sen</code> / GBV
NPS	Post-completion survey	≥45	Reviews + survey
Ops	Webhook failure rate	<0.5%	<code>payments.status='failed'</code> + Sentry
Ops	Review fraud rate	~0%	Audit: <code>reviews</code> without valid booking
Engineering	Build + typecheck + lint clean	100% of merges	CI <code>quality-gate.yml</code>

Leading indicators tracked via `src/server/agnost.ts` + `src/lib/agnost-client.ts`: `trackArtistView`, `trackSearch`, `trackBookingForm`, bookings, payments.

10. Constraints, Assumptions & Dependencies

10.1 CONSTRAINTS

ID	CONSTRAINT
C-01	Must run on Vercel (serverless); no long-running processes — crons via <code>VERCEL_CRON</code> or external scheduler.
C-02	<code>DATABASE_URL</code> (PostgreSQL) in production; <code>node:sqlite</code> fallback for local/tests only — never in prod (<code>SKIP_DB_GUARD</code> only for debugging).
C-03	Billplz is the sole gateway at launch; no multi-PSP abstraction needed beyond <code>provider='billplz' 'dev'</code> .
C-04	Single deployable (Next.js); <code>output:"standalone"</code> for Docker parity.
C-05	Node ≥ 22 required (for <code>node:sqlite</code> DatabaseSync).

10.2 ASSUMPTIONS

ID	ASSUMPTION	VALIDATION PLAN
A-01	RM50 flat deposit is acceptable to clients	A/B price sensitivity after 100 bookings
A-02	10% commission + waiver is acceptable to artists	Artist interviews + churn watch
A-03	24h quotation window balances urgency vs deliberation	Funnel analysis (expiry vs payment)
A-04	T-3d balance due date is enforceable (reminder only for now)	Add dunning cron in v2.1 if delinquency >10%
A-05	Email is sufficient for transactional comms at launch	Add WhatsApp in v2.1 if open rate <40%

10.3 DEPENDENCIES

DEPENDENCY	TYPE	RISK IF UNAVAILABLE
Billplz API + Collection ID	External	<code>activePaymentProvider()</code> falls back to <code>dev</code> (no real money); feature-flagged warning
<code>NEXT_PUBLIC_SUPABASE_URL</code> + anon/service keys	External (storage/images)	Catalog images degrade to placeholder; admin Supabase pages gated
RESEND/POSTMARK/BREVO	External	Falls back to <code>email_outbox</code> (no real send, <code>dev</code> provider) — flagged in logs
Upstash Redis	Optional	In-memory rate limiting (single-instance degradation)
Vercel Blob / S3	Optional	Image uploads degrade; local fallback

11. Compliance & Legal

AREA	REQUIREMENT	IMPLEMENTATION
PDPA 2010 (Malaysia)	Consent captured; purpose limitation; access & correction; retention limit	<code>registerSchema.consent</code> , <code>users.consent+consent_timestamp</code> , <code>GET /api/me/export</code> , <code>DELETE /api/me</code> , <code>docs/PDPA_RETENTION_GUIDELINES.md</code>
Consumer Protection (KPDN)	Price transparency; refund policy disclosed	Quotation breakdown, invoice HTML/PDF, non-refundable deposit disclosed at pay-fee
Data Retention	Bookings/payouts retained for audit; PII deletable on request	Retention guideline doc; <code>DELETE /api/me</code> cascades <code>ON DELETE CASCADE</code>
Security disclosures	Headers, CSP, password handling	<code>next.config.ts</code> headers (<code>X-Frame-Options:DENY</code> , <code>X-Content-Type-Options:nosniff</code> , CSP with nonce), scrypt + pepper, httpOnly cookies
Financial	Settlement auditability	<code>payouts</code> + <code>admin_audit_log</code> immutable log

12. Risks & Mitigations

ID	RISK	LIKELIHOOD	IMPACT	MITIGATION
R-01	Double-booking same slot (race)	Medium	High	Partial unique index <code>uq_bookings_slot</code> + optimistic 409; never trust client price
R-02	Webhook forgery (fake paid)	Medium	Critical	<code>verifyBillplzSignature()</code> timing-safe before state change; apiKey as HMAC secret
R-03	Lost-update on rating	Low	Medium	<code>blendAggregate()</code> single SQL (no read-modify-write) + <code>ROUND(...,2)</code>
R-04	SQLite used in prod by misconfig	Low	Critical	<code>getDb()</code> throws in prod when <code>DATABASE_URL</code> missing (unless <code>SKIP_DB_GUARD</code>)
R-05	Email provider downtime	Medium	Medium	Fallback to dev outbox; retry queue <code>email_retries</code> max 3, <code>next_retry</code> backoff
R-06	Artist churn (take-rate too high)	Medium	High	Waiver threshold; configurable <code>platform_settings</code> ; artist dashboard transparency
R-07	Low conversion accepted→confirmed	Medium	Medium	24h expiry urgency + re-quote; analytics on funnel
R-08	PG pool exhaustion	Low	High	<code>PG_MAX</code> , <code>PG_CONNECTION_TIMEOUT_MS</code> , <code>PG_IDLE_TIMEOUT_MS</code> tunable; <code>idleTimeoutMillis</code> for Neon/serverless

13. Roadmap & Release Strategy

PHASE	MILESTONE	SCOPE	GATE
0 — Foundation	DB facade, auth, catalog	Tables, session, seeding, filters	Typecheck + tests pass
1 — Booking Loop	Request → accept → quote → fee → confirm	Bookings, quotations, payments (dev)	E2E (register→book→dashboard) green
2 — Money Hardening	Billplz, webhook, payouts, invoices	<code>verifyBillplzSignature</code> , <code>handlePaymentPaid</code> , <code>payouts</code>	Manual Billplz sandbox test + audit
3 — Comms	Chat, emails, preferences	SSE stream, <code>booking-emails</code> , <code>email_preferences</code>	Mail trap + load test
4 — Admin	Ops panel + audit	<code>/admin/*</code> , <code>admin_audit_log</code> , <code>platform_settings</code>	Admin UAT + seed-admin
5 — Launch	KL+Selangor, 10 MUAs	Seed catalog, health check, deploy	<code>npm run env:check</code> + <code>db:migrate/seed</code>

Post-launch (v2.1+): studio-native bookings, balance dunning cron, WhatsApp, product e-commerce, native loyalty.

14. Acceptance Criteria (Business Level)

☐

A guest can browse artists/studios and filter without authentication.

☐

A customer can register (with PDPA consent), verify email, and log in; sessions last 7 days.

☐

An artist can claim a profile and see only their bookings; `GET /api/bookings` is scoped.

☐

Double-booking a live slot returns a friendly 409 (never 500) and the unique index prevents the anomaly.

☐

Quotations are 24h expiry, re-quotable, and total is server-computed; expired ones are auto-marked.

☐

Deposits confirm bookings only via verified webhook (Billplz) or dev auto-settlement (test).

☐

Commission math matches `computeCommission()` spec with waivers and clamped rate.

☐

Payouts become settleable 24h after event date and are admin-settleable with audit.

☐

Reviews are possible only after `completed`, once per booking, with atomic rating update.

☐

Per-booking chat works live (SSE) and persists.

☐

Admin actions are audit-logged; non-admin cannot access `/admin` or `/api/admin/*`.

☐

PDPA export/delete work as documented; retention guidelines are followed.

☐

Build, lint, typecheck, tests (Vitest + Playwright), and coverage ≥80% all pass in CI.

15. Appendix

15.1 REFERENCES

- `AGENTS.md` — project overview, architecture patterns, security considerations.
- `docs/ARCHITECTURE.md` — single-backend data flow, money logic ownership.
- `docs/CODEBASE-AUDIT-FINAL.md` — enterprise-readiness gaps closed.
- `src/lib/data.ts` — constants (`MALAYSIA_STATES` , `BRIDAL_EVENTS` , `NON_BRIDAL_EVENTS` , `SEED_ARTISTS/STUDIOS`).
- `src/server/db.ts` — `PG_SCHEMA` / `SQLITE_SCHEMA` , `DbFacade` , migration logic.

15.2 GLOSSARY (SHORT)

See `docs/09-Glossary.md` for full glossary. Key terms: **Deposit**, **Balance**, **Quotation**, **Payout**, **Settlement**, **Claim**, **Slot**.

15.3 DOCUMENT CONTROL

VERSION	DATE	AUTHOR	CHANGE
1.0	2026-08-10	Product	Initial BRS covering booking→fee
2.0	2026-08-29	Product+Eng	Added hybrid deposit/ balance, payouts/ commission, admin, compliance

End of BRS — see `02-SRS-Software-Requirements-Specification.md` for system decomposition and `03-FRS-Functional-Requirements-Specification.md` for per-feature functional detail.